Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет информационных технологий и программирования
Кафедра прикладной информатики

ДОМАШНЕЕ ЗАДАНИЕ №6

Настройка GitLab CI/CD для автоматического развёртывания бэкенд-приложения

Выполнил: студент группы К3341
Кижваткин Н.М.

Санкт-Петербург
2025

1. ВВЕДЕНИЕ

Настоящий отчёт посвящён настройке системы непрерывной интеграции и доставки (CI/CD) для проекта Callmentor с использованием GitLab CI. CI/CD автоматизирует процесс сборки Docker-образов и их публикации в Container Registry, что обеспечивает стандартизацию развёртывания и снижает вероятность человеческих ошибок.

2. СТРУКТУРА CI/CD ПАЙПЛАЙНА

2.1. Конфигурационный файл

CI/CD пайплайн описан в файле .gitlab-ci.yml в корне репозитория. Файл использует механизм include для подключения переиспользуемой конфигурации docker-compose.gitlab-ci.yml из корпоративного репозитория devops/pipelines. Это позволяет стандартизировать процесс сборки образов по всем проектам компании Metamentor и централизованно управлять конфигурацией CI/CD.

2.2. Стадии пайплайна

Пайплайн включает одну стадию: build — сборка Docker-образов и их публикация в Harbor Registry. Каждый job extends базовый шаблон .docker-builder, определённый в подключаемом файле.

3. JOB-ОПИСАНИЯ

Каждый сервис имеет отдельный CI-job:

Таблица 1 — CI/CD jobs системы Callmentor

Job	Репозиторий Harbor	Build Context
build_nginx	callmentor/nginx	./nginx
build_frontend	callmentor/frontend	./callmentor-frontend
build_gateway	callmentor/gateway	./gateway/api
build_ml_service	callmentor/ml-service	./ml-service
build_ml_llm	callmentor/ml-llm	./ml-llm
build_ml_asr	callmentor/ml-asr	./ml-asr

3.1. Конфигурация переменных job

Каждый job передаёт набор переменных шаблону .docker-builder:

- TARGET_BUILD_CONTEXT — директория с контекстом сборки Docker-образа.
- TARGET_DOCKERFILE_PATH — путь к Dockerfile относительно контекста.

– TARGET_REGISTRY_REPOSITORY — путь в Harbor Registry для публикации образа.
– TARGET_CACHE_FROM_REFERENCE — ссылка на кэш-образ (ускоряет сборку за счёт слоёв предыдущей сборки).
– EXTRA_IMAGE_TAGS — дополнительные теги образа (определяются из $PRJ_EXTRA_IMAGE_TAGS).

4. МЕХАНИЗМ КЭШИРОВАНИЯ

Для ускорения сборки Docker-образов используется механизм кэширования слоёв. Переменная TARGET_CACHE_FROM_REFERENCE указывает на образ из Registry, слои которого будут использованы как кэш при следующей сборке. Формат: $REGISTRY_HOST/callmentor/<service>:$PRJ_CACHE_FROM_REFERENCE_TAG.
Типично PRJ_CACHE_FROM_REFERENCE_TAG указывает на тег main или latest от предыдущего успешного запуска пайплайна.

5. ПЕРЕМЕННЫЕ GITLAB CI

Пайплайн использует следующие переменные окружения GitLab:

Таблица 2 — Переменные GitLab CI/CD

Переменная	Назначение
REGISTRY_HOST	Хост Harbor Registry (harbor.metamentor.pro)
PRJ_CACHE_FROM_REFERENCE_TAG	Тег кэш-образа (обычно main)
PRJ_EXTRA_IMAGE_TAGS	Дополнительные теги (например, latest)
IMAGE_TAG	Основной тег образа (SHA коммита или тег релиза)
CI_REGISTRY_USER	Логин для Harbor (auto-managed GitLab)
CI_REGISTRY_PASSWORD	Пароль для Harbor (auto-managed GitLab)

6. ПРОЦЕСС CONTINUOUS DELIVERY

Полный цикл CI/CD для системы Callmentor:

1. Разработчик делает git push в ветку main.
2. GitLab CI автоматически запускает пайплайн для изменившихся сервисов.
3. Шаблон .docker-builder выполняет: docker build с указанным контекстом, тегирование образа, docker push в Harbor Registry.

4. После успешной публикации образов команда запускает развёртывание на сервере: docker compose -f docker/compose.prod.yaml pull && docker compose -f docker/compose.prod.yaml up -d.
5. Теги образов (IMAGE_TAG) управляются через переменные CI, обеспечивая версионирование и возможность отката.

6.1. Стратегия тегирования образов

Образы тегируются несколькими тегами для удобства управления:

– SHA коммита (например, abc123f) — уникальная идентификация каждой сборки, обеспечивает воспроизводимость.
– Имя ветки (main) — всегда указывает на актуальный образ ветки.
– latest — указывает на последнюю стабильную версию.

7. ПРЕИМУЩЕСТВА CI/CD ДЛЯ ПРОЕКТА

Настроенный CI/CD обеспечивает следующие преимущества:

– Стандартизация: каждый образ собирается по единому воспроизводимому процессу.
– Скорость: параллельная сборка 6 образов сокращает время до развёртывания.
– Кэширование: повторное использование слоёв Docker ускоряет инкрементальные сборки.
– Версионирование: каждый коммит имеет соответствующий образ в Registry.
– Откат: при проблемах можно быстро вернуться к предыдущей версии образа.
– Прозрачность: статус сборки виден в интерфейсе GitLab.

8. ВЫВОДЫ

Настроен GitLab CI/CD пайплайн для автоматической сборки и публикации Docker-образов системы Callmentor. Пайплайн охватывает все 6 сервисов с собственными Dockerfile. Использование централизованного шаблона .docker-builder обеспечивает согласованность процесса сборки. Механизм кэширования слоёв ускоряет инкрементальные сборки. В сочетании с продакшн конфигурацией Docker Compose, CI/CD пайплайн формирует полноценный цикл DevOps для системы Callmentor.